

# How to make Terse Notation soar with Class Template Argument Deduction

Terse concept notation along the lines of [P1141R1](#) has the potential to powerfully make generic programming look more like “regular programming”. As it is important for the initial release of concepts and terse notation to make a good first impression, this paper discusses how Class Template Argument Deduction can pitch in and help make the release of terse notation as successful as possible. as illustrated by the following example (note that Curve is a concept).

| C++17 + P1141R1                                                                                                                                                                                                                                                                                                                                                                                                          | Proposed                                                                                                                                                                         |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>auto f(Curve auto &amp;c) -&gt; pair&lt;decltype(c), pmr::vector&lt;typename decltype(c.ctrl_pts)::value_type&gt;&gt; {     Point&lt;decltype(c.x_min())&gt; top_left{c.x_min(), c.y_max()};     screen-&gt;line&lt;decltype(p.x)&gt;(p, {c.x_min(), c.y_max()});     pmr::vector&lt;typename decltype(c.ctrl_pts)::value_type&gt; cp(c.ctrl_pts.begin(), c.ctrl_pts.end(), mem_res);     return {c, cp }; };</pre> | <pre>pair auto f(Curve auto &amp;c, basic_c {     Point top_left{c.x_min(), c.y_ma screen-&gt;line(top_left, {c.x_min( pmr::vector cp(c.ctrl_pts.begin( return {c, cp}; };</pre> |

What is going on here? Since function template arguments in terse notation often do not have named types, the easier it is to create new objects from existing objects without having to explicitly refer to their types, the more natural code will be. Since Class Template Argument Deduction provides inferred creation of constrained objects just as concepts provide inferred usage of constrained objects (thereby addressing complementary problems), it can really help in such situations as the above example shows. For a stunning example of how much robust Class Template Argument Deduction can enhance concepts code, see the argument deduction [example](#) near the end of this paper.

While C++17 Class Template Argument Deduction can often be used in such situations (e.g., if the `pmr::vector` above had been an ordinary vector), the example above was crafted to illustrate the value of potential improvements to Class Template Argument Deduction, such as return type deduction below, and aggregate and type alias deduction. Such basic "filling of holes" would help a lot as illustrated in the example, and is covered in [P1021R1](#) and [P1167R0](#). The remainder of this paper focuses specifically on aligning Class Template Argument Deduction Inferencing with Constrained Type Inferencing.

## The proposal

Our proposal is simply that wherever P1141R1 allows a Concept-constrained auto, a class template can be used as well with the same rules for independent resolution and forwarding references. Of course, if a different approach from P1141 is adopted for constrained declarations, this proposal should follow that. Likewise, we do not take a position on where auto should be required, simply that the same rules should apply to CTAD inferencing and concept inferencing to best maintain consistency between concepts and types.

Let us look at some particular cases.

### Ordinary declarations

In P1141R1, it is mentioned that in

```
Constraint auto f1();
```

it should be possible to omit the auto to get

```
Constraint f1();
```

As justification, P1141R1 specifically refers to Class Template Argument Deduction allowing

```
std::tuple x = foo();
```

By the same reasoning, it makes sense for us to be consistent in the other direction and allow

```
std::tuple auto x = foo();
```

Otherwise, the programmer has to remember a bunch of special-cased rules as to which kinds of deduced declarations can accept an auto or not.

### Return type deduction

Likewise, just like one can say

```
Constraint auto f1();
```

one should also be able to say

```
tuple auto f1();
```

to make explicit that constrained type deduction is taking place (In this context, we are using the phrase “constrained type deduction” to refer to the fact that the deduced type is given more specifically than just a lone auto). The example given at the top of this paper of `f` returning

```
pair auto
```

instead of

```
pair<decltype(c), pmr::vector<typename decltype(c.ctrl_pts)::value_type>>
```

illustrates just how much this can simplify code.

Notes:

- As with normal return type deduction, the function needs to have a return statement, and all return statements must generate the same type. In [P1021R0](#), it was proposed that the return statements could have different types, but this is no longer proposed after feedback from compiler vendors.
- This proposal also follows the principal that returning a value should behave like initialization:

```
tuple t = {1, "foo"};
tuple auto g() { return {1, "foo"}; }
```

Argument deduction

As P1141R0 allows us to say

```
void f1(Constraint auto x);
```

one should also be able to indicate that a declaration has its arguments deduced by

```
void f1(tuple auto x);
```

This is not only more consistent, but we think it can have a huge impact on facilitating terse notation as now almost any function template could be written using natural function notation as shown by the following example (which also demonstrates the value of partially-specialized template argument lists as proposed in P1021R0) extension

| C++17 + P1141R1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     | Pr                                                                                                                                                                                                                                                                                                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>// We need an explicit template here because of confusing mix of both class template and concept arguments template&lt;class CharT, class Traits&gt; auto f(Curve auto &amp;c, basic_ostream&lt;CharT, Traits&gt; logger) -&gt; pair&lt;decltype(c), pmr::vector&lt;typename decltype(c.ctrl_pts)::value_type&gt;&gt; {     Point&lt;decltype(c.x_min())&gt; top_left{c.x_min(), c.y_max()};     screen-&gt;line&lt;decltype(p.x)&gt;(p, {c.x_min(), c.y_max()});     logger &lt;&lt; "Drew line\n";      pmr::vector&lt;typename decltype(c.ctrl_pts)::value_type&gt; cp(c.ctrl_pts.begin(), c.ctrl_pts.end(), mem_res);     logger &lt;&lt; "Got control points: ";     // Hard to specify ostream_iterator template args     copy(cp.begin(), cp.end(),         ostream_iterator&lt;decltype(cp[0]),             typename decltype(logger)::char_type,             typename decltype(logger)::traits_type&gt;(logger, ", "));     return {c, cp }; };</pre> | <pre>// "auto" constraints consistently pair auto f(Curve auto &amp;c, basic_c {     Point auto top_left{c.x_min(), c screen-&gt;line(top_left, {c.x_min( logger &lt;&lt; "Drew line\n";      pmr::vector cp(c.ctrl_pts.begin( logger &lt;&lt; "Got control points: " // See P1021R1 for partially-spe copy(cp.begin(), cp.end(),     ostream_iterator&lt;decltype(c return {c, cp}; };</pre> |

We think things like the above example will be very common, and, without this proposal, terse notation will not be applicable to many function templates.

Specialization

As Zhihao Yuan notes, the partially template argument list support proposed in P1167R0 naturally supports the useful alignment

```
tuple<Copyable> t(1);
```